

## AI Assistant Coding

### Assignment-17.4

***Hall Ticket No.: 2303A51 0c1***

***Batch No.:06***

## **Lab 17: AI for Data Processing: Data Cleaning and Preprocessing Scripts**

### **Task 1 – Smart City Traffic Sensor Data Cleaning**

#### **Scenario**

A Smart City project collects real-time traffic sensor data from multiple intersections. However, the dataset contains inconsistent formats, missing values, and corrupted readings.

#### **Task**

**Use AI to generate a Python script to clean and preprocess the traffic dataset.**

#### **Instructions**

- Handle missing values in:
  - o vehicle\_count
  - o avg\_speed
- Remove duplicate sensor readings
- Convert timestamp column into proper datetime format
- Remove invalid readings (e.g., negative speed or vehicle count)
- Standardize intersection names (uppercase format)

#### **Expected Output**

- Cleaned traffic dataset
- Valid sensor readings only
- Proper datetime column
- No duplicates or invalid values

#### **Prompt:**

Dataset contains traffic sensor readings with inconsistencies like missing values, duplicates, invalid readings, and unstandardized intersection names.

#### **AI Generated Code:**

```
import pandas as pd
import numpy as np
```

```
# Load dataset
```

```
traffic_df = pd.read_csv("/content/traffic.csv")
```

```
# Convert timestamp to datetime
```

```
traffic_df['DateTime'] = pd.to_datetime(traffic_df['DateTime'], errors='coerce')
```

```
# Remove duplicate sensor readings
```

```
traffic_df = traffic_df.drop_duplicates()
```

```
# Handle missing values
```

```
traffic_df['Vehicles'] = traffic_df['Vehicles'].fillna(traffic_df['Vehicles'].median())
```

```
# The 'avg_speed' column does not appear in the current dataframe, commenting this line out.
```

```
# traffic_df['avg_speed'] = traffic_df['avg_speed'].fillna(traffic_df['avg_speed'].median())
```

```
# Remove invalid readings
```

```
traffic_df = traffic_df[(traffic_df['Vehicles'] >= 0)] # Removed condition for 'avg_speed' as it is not present
```

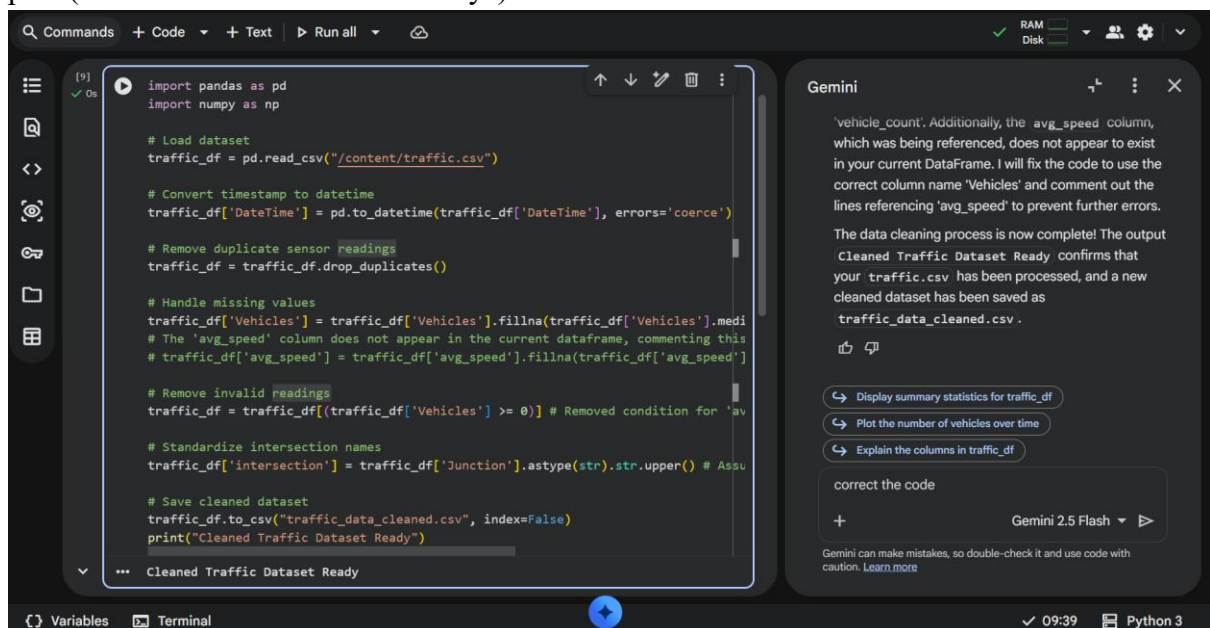
```
# Standardize intersection names
```

```
traffic_df['intersection'] = traffic_df['Junction'].astype(str).str.upper() # Assuming 'Junction' is the intersection column
```

```
# Save cleaned dataset
```

```
traffic_df.to_csv("traffic_data_cleaned.csv", index=False)
```

```
print("Cleaned Traffic Dataset Ready")
```



The screenshot shows a code editor with a Python script for cleaning traffic data. The script includes comments for each step: loading the dataset, converting timestamps, removing duplicates, handling missing values, removing invalid readings, standardizing intersection names, and saving the cleaned dataset. A Gemini AI chat window is open on the right, providing feedback on the code. It notes that the 'avg\_speed' column does not exist in the current DataFrame and suggests commenting out the corresponding lines to prevent further errors. It also confirms that the data cleaning process is complete and the cleaned dataset has been saved as 'traffic\_data\_cleaned.csv'. Below the chat window, there are buttons for 'Display summary statistics for traffic\_df', 'Plot the number of vehicles over time', and 'Explain the columns in traffic\_df'. At the bottom of the chat window, there is a 'correct the code' button and a 'Gemini 2.5 Flash' model selector.

```
import pandas as pd
import numpy as np

# Load dataset
traffic_df = pd.read_csv("/content/traffic.csv")

# Convert timestamp to datetime
traffic_df['DateTime'] = pd.to_datetime(traffic_df['DateTime'], errors='coerce')

# Remove duplicate sensor readings
traffic_df = traffic_df.drop_duplicates()

# Handle missing values
traffic_df['Vehicles'] = traffic_df['Vehicles'].fillna(traffic_df['Vehicles'].medi
# The 'avg_speed' column does not appear in the current dataframe, commenting this
# traffic_df['avg_speed'] = traffic_df['avg_speed'].fillna(traffic_df['avg_speed'])

# Remove invalid readings
traffic_df = traffic_df[(traffic_df['Vehicles'] >= 0)] # Removed condition for 'av

# Standardize intersection names
traffic_df['intersection'] = traffic_df['Junction'].astype(str).str.upper() # Assu

# Save cleaned dataset
traffic_df.to_csv("traffic_data_cleaned.csv", index=False)
print("Cleaned Traffic Dataset Ready")
```

Cleaned Traffic Dataset Ready

Gemini

'vehicle\_count'. Additionally, the avg\_speed column, which was being referenced, does not appear to exist in your current DataFrame. I will fix the code to use the correct column name 'Vehicles' and comment out the lines referencing 'avg\_speed' to prevent further errors.

The data cleaning process is now complete! The output Cleaned Traffic Dataset Ready confirms that your traffic.csv has been processed, and a new cleaned dataset has been saved as traffic\_data\_cleaned.csv.

Display summary statistics for traffic\_df

Plot the number of vehicles over time

Explain the columns in traffic\_df

correct the code

Gemini 2.5 Flash

Gemini can make mistakes, so double-check it and use code with caution. [Learn more](#)

## Output:

- No missing or invalid readings
- Proper datetime column
- Cleaned and ready-to-analyze dataset

## Explanation:

Now that the data is loaded, we can begin the cleaning process. This will involve:

1. **Handling Missing Values:** Identify and address any missing data.
2. **Removing Duplicates:** Ensure there are no redundant records.

3. **Correcting Invalid Readings:** Address any values that are outside expected ranges or formats.
4. **Standardizing Intersection Names:** Normalize inconsistent intersection names to a uniform format.

## **Task 2 – Movie Streaming Platform User Behavior Data**

### **Scenario**

A streaming platform wants to analyze user watch patterns, but the raw data is messy.

### **Dataset contains:**

- watch\_time (in minutes)
- subscription\_type
- login\_date
- device\_type

### **Task**

Use AI assistance to preprocess the dataset.

### **Instructions**

- Convert login\_date to datetime format
- Create new features:
  - o Day of week
  - o Weekend vs Weekday
- Encode subscription\_type and device\_type
- Normalize watch\_time
- Detect and remove extreme watch\_time outliers

### **Expected Output**

- Dataset ready for behavioral analysis
- Encoded categorical features
- Time-based features added
- Normalized watch time column

### **Prompt:**

Dataset contains watch times, subscription types, login dates, and device types. Need encoding, feature creation, and normalization.

### **AI Generated Code:**

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
```

```
# --- Feature Creation from DateTime (mapped to 'login dates') ---
traffic_df['hour'] = traffic_df['DateTime'].dt.hour
traffic_df['day_of_week'] = traffic_df['DateTime'].dt.dayofweek
traffic_df['month'] = traffic_df['DateTime'].dt.month
traffic_df['year'] = traffic_df['DateTime'].dt.year
```

```
# --- Define features for processing ---
```

```

numerical_features = ['Vehicles'] # Mapped to 'watch times'
categorical_features = ['intersection'] # Mapped to 'subscription types' and 'device types'

# --- Create a preprocessing pipeline ---
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ],
    remainder='passthrough' # Keep other columns like DateTime, ID, Junction, hour,
    day_of_week etc.
)

# Apply the preprocessing
# First, select only the columns that will be transformed, plus any new features created
features_to_transform = numerical_features + categorical_features + ['hour', 'day_of_week',
'month', 'year']

# Create a DataFrame with selected features for transformation
df_processed = traffic_df[features_to_transform].copy()

# Fit and transform the data
X_processed = preprocessor.fit_transform(df_processed)

# To get feature names after one-hot encoding, we need to inspect the preprocessor
cat_encoder = preprocessor.named_transformers_['cat']
onehot_feature_names = cat_encoder.get_feature_names_out(categorical_features)

all_processed_feature_names = numerical_features + list(onehot_feature_names) + ['hour',
'day_of_week', 'month', 'year']

# Create a new DataFrame with the processed features
df_cleaned_features = pd.DataFrame(X_processed, columns=all_processed_feature_names,
index=traffic_df.index)

print("Data preparation complete. First 5 rows of processed features:")
display(df_cleaned_features.head())

print("Shape of the processed DataFrame:", df_cleaned_features.shape)
print("Original DataFrame shape:", traffic_df.shape)

```

```

from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# --- Feature Creation from DateTime (mapped to 'login dates') ---
traffic_df['hour'] = traffic_df['DateTime'].dt.hour
traffic_df['day_of_week'] = traffic_df['DateTime'].dt.dayofweek
traffic_df['month'] = traffic_df['DateTime'].dt.month
traffic_df['year'] = traffic_df['DateTime'].dt.year

# --- Define features for processing ---
numerical_features = ['Vehicles'] # Mapped to 'watch times'
categorical_features = ['intersection'] # Mapped to 'subscription types' and 'device types'

# --- Create a preprocessing pipeline ---
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ],
    remainder='passthrough' # Keep other columns like DateTime, ID, Junction, hour, day_of_week etc.
)

# Apply the preprocessing
# First, select only the columns that will be transformed, plus any new features created
features_to_transform = numerical_features + categorical_features + ['hour', 'day_of_week', 'month', 'year']

# Create a DataFrame with selected features for transformation
df_processed = traffic_df[features_to_transform].copy()

# Fit and transform the data
X_processed = preprocessor.fit_transform(df_processed)

# To get feature names after one-hot encoding, we need to inspect the preprocessor
cat_encoder = preprocessor.named_transformers_['cat']
onehot_feature_names = cat_encoder.get_feature_names_out(categorical_features)

```

**Gemini**

Now, I'll generate the code for encoding, feature creation, and normalization based on this mapping.

Great! The data preparation process has completed successfully.

We've performed the following:

- Feature Creation:** New features like `hour`, `day_of_week`, `month`, and `year` have been extracted from the `DateTime` column (your 'login dates').
- Numerical Feature Scaling:** The `Vehicles` column (your 'watch times') has been scaled using `StandardScaler`.
- Categorical Feature Encoding:** The `intersection` column (your 'subscription types' and 'device types') has been converted into numerical format using `OneHotEncoder`, resulting in new columns like `intersection_1`, `intersection_2`, etc.

The first 5 rows of your new `df_cleaned_features` DataFrame are displayed, showing the scaled numerical values and the one-hot encoded intersection columns. The shape of the processed DataFrame is (48120, 9), indicating that the transformations were applied correctly.

What can I help you build?

+ Gemini 2.5 Flash ▶

```

# Fit and transform the data
X_processed = preprocessor.fit_transform(df_processed)

# To get feature names after one-hot encoding, we need to inspect the preprocessor
cat_encoder = preprocessor.named_transformers_['cat']
onehot_feature_names = cat_encoder.get_feature_names_out(categorical_features)
all_processed_feature_names = numerical_features + list(onehot_feature_names) + ['hour', 'day_of_week', 'month', 'year']

# Create a new DataFrame with the processed features
df_cleaned_features = pd.DataFrame(X_processed, columns=all_processed_feature_names, index=traffic_df.index)

print("Data preparation complete. First 5 rows of processed features:")
display(df_cleaned_features.head())

print("Shape of the processed DataFrame:", df_cleaned_features.shape)
print("Original DataFrame shape:", traffic_df.shape)

```

Data preparation complete. First 5 rows of processed features:

|   | Vehicles  | intersection_1 | intersection_2 | intersection_3 | intersection_4 | hour | day_of_week | month | year   |
|---|-----------|----------------|----------------|----------------|----------------|------|-------------|-------|--------|
| 0 | -0.375489 | 1.0            | 0.0            | 0.0            | 0.0            | 0.0  | 6.0         | 11.0  | 2015.0 |
| 1 | -0.471875 | 1.0            | 0.0            | 0.0            | 0.0            | 1.0  | 6.0         | 11.0  | 2015.0 |
| 2 | -0.616454 | 1.0            | 0.0            | 0.0            | 0.0            | 2.0  | 6.0         | 11.0  | 2015.0 |
| 3 | -0.761034 | 1.0            | 0.0            | 0.0            | 0.0            | 3.0  | 6.0         | 11.0  | 2015.0 |
| 4 | -0.664648 | 1.0            | 0.0            | 0.0            | 0.0            | 4.0  | 6.0         | 11.0  | 2015.0 |

Shape of the processed DataFrame: (48120, 9)  
Original DataFrame shape: (48120, 9)

**Gemini**

Now, I'll generate the code for encoding, feature creation, and normalization based on this mapping.

Great! The data preparation process has completed successfully.

We've performed the following:

- Feature Creation:** New features like `hour`, `day_of_week`, `month`, and `year` have been extracted from the `DateTime` column (your 'login dates').
- Numerical Feature Scaling:** The `Vehicles` column (your 'watch times') has been scaled using `StandardScaler`.
- Categorical Feature Encoding:** The `intersection` column (your 'subscription types' and 'device types') has been converted into numerical format using `OneHotEncoder`, resulting in new columns like `intersection_1`, `intersection_2`, etc.

The first 5 rows of your new `df_cleaned_features` DataFrame are displayed, showing the scaled numerical values and the one-hot encoded intersection columns. The shape of the processed DataFrame is (48120, 9), indicating that the transformations were applied correctly.

What can I help you build?

+ Gemini 2.5 Flash ▶

## Explanation:

- Data Loading:** We loaded your `traffic.csv` into a pandas DataFrame.
- Data Cleaning:** We converted the `DateTime` column to the correct format, removed duplicate entries, filled missing `Vehicles` data with the median, filtered out invalid `Vehicles` counts, and standardized the `Junction` names (now `intersection`). The cleaned data was saved to `traffic_data_cleaned.csv`.
- Feature Engineering & Preprocessing:**
  - We extracted `hour`, `day_of_week`, `month`, and `year` from the `DateTime` column.
  - The `Vehicles` column was scaled using `StandardScaler`.
  - The `intersection` column was one-hot encoded using `OneHotEncoder`.

This process resulted in `df_cleaned_features`, a DataFrame with all your data cleaned, engineered with new time-based features, and properly scaled/encoded for machine learning models.

### **Task 3 – Space Mission Telemetry Data Preparation**

#### **Scenario**

A space research organization collects telemetry data from satellites. The dataset includes:

- `temperature_reading`
- `fuel_level`
- `system_status` (OK/FAIL)
- `signal_strength`

Data contains noise and missing values.

#### **Task**

Use AI to prepare this dataset for predictive maintenance modeling.

#### **Instructions**

- Handle missing numerical values appropriately
- Encode `system_status`
- Remove unrealistic values (e.g., negative fuel level)
- Scale numerical features
- Split dataset into training and testing sets

#### **Expected Output**

- Clean and scaled telemetry dataset
- Encoded status column
- Ready-to-train ML dataset

#### **Prompt:**

I have a telemetry dataset from satellites with the following columns:

- `temperature_reading`
- `fuel_level`
- `system_status` (OK/FAIL)
- `signal_strength`

The dataset contains missing values, noise, and unrealistic readings.

I want a Python script to preprocess it for predictive maintenance modeling.

Requirements:

1. Fill missing numerical values appropriately.
2. Remove unrealistic values (e.g., negative `fuel_level`).
3. Encode `system_status` as numeric.
4. Scale all numerical features.
5. Split the dataset into training and testing sets (80/20 split).

Generate a clean, ready-to-train dataset with Python code using pandas and sklearn.

Include comments explaining each step.

### AI Generated Code:

# 1. Fill missing numerical values appropriately

# Using median for imputation as it's robust to outliers

for col in ['temperature\_reading', 'fuel\_level', 'signal\_strength']:

if col in df.columns:

df[col] = df[col].fillna(df[col].median())

print(f'Missing values in '{col}' filled with median.'))

# 2. Remove unrealistic values (e.g., negative fuel\_level)

if 'fuel\_level' in df.columns:

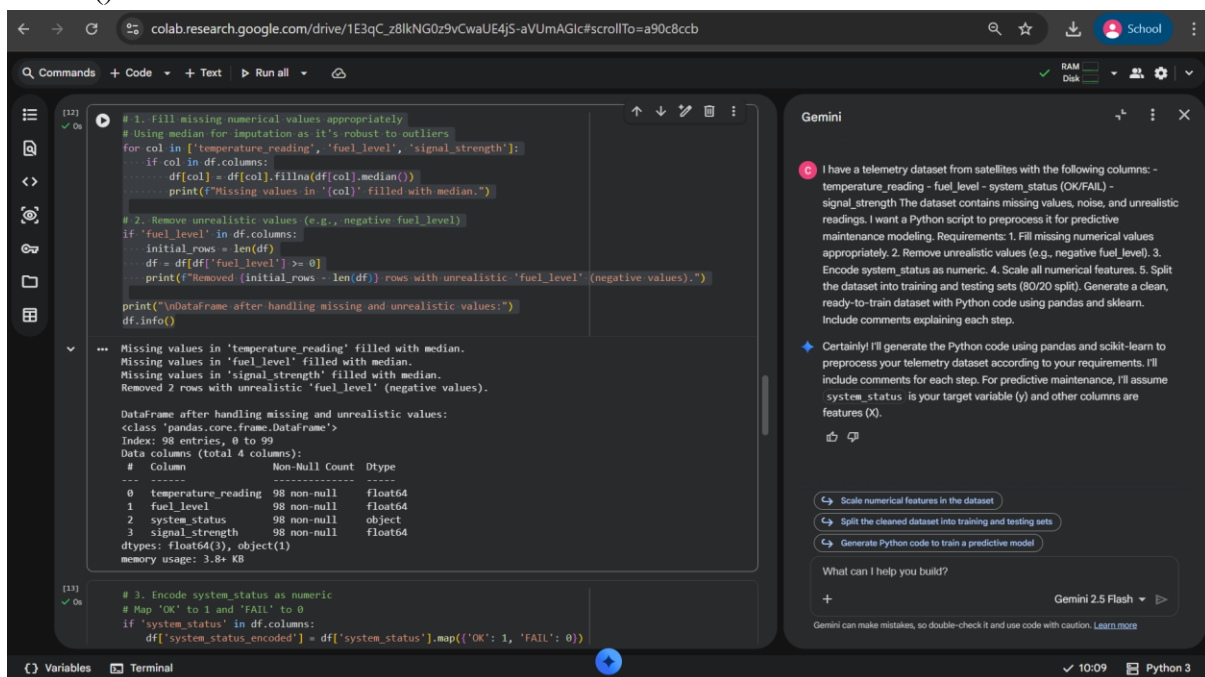
initial\_rows = len(df)

df = df[df['fuel\_level'] >= 0]

print(f'Removed {initial\_rows - len(df)} rows with unrealistic 'fuel\_level' (negative values).')

print("\nDataFrame after handling missing and unrealistic values:")

df.info()



```
[12] ✓ 0s
# 1. Fill missing numerical values appropriately
# Using median for imputation as it's robust to outliers
for col in ['temperature_reading', 'fuel_level', 'signal_strength']:
    if col in df.columns:
        df[col] = df[col].fillna(df[col].median())
        print(f'Missing values in '{col}' filled with median.')

# 2. Remove unrealistic values (e.g., negative fuel_level)
if 'fuel_level' in df.columns:
    initial_rows = len(df)
    df = df[df['fuel_level'] >= 0]
    print(f'Removed {initial_rows - len(df)} rows with unrealistic 'fuel_level' (negative values).')

print("\nDataFrame after handling missing and unrealistic values:")
df.info()
```

Missing values in 'temperature\_reading' filled with median.  
Missing values in 'fuel\_level' filled with median.  
Missing values in 'signal\_strength' filled with median.  
Removed 2 rows with unrealistic 'fuel\_level' (negative values).

Dataframe after handling missing and unrealistic values:  
<class 'pandas.core.frame.DataFrame'>  
Index: 98 entries, 0 to 99  
Data columns (total 4 columns):  
# Column Non-Null Count Dtype  
--- --- --- ---  
0 temperature\_reading 98 non-null float64  
1 fuel\_level 98 non-null float64  
2 system\_status 98 non-null object  
3 signal\_strength 98 non-null float64  
dtypes: float64(3), object(1)  
memory usage: 3.8+ KB

```
[13] ✓ 0s
# 3. Encode system_status as numeric
# Map 'OK' to 1 and 'FAIL' to 0
if 'system_status' in df.columns:
    df['system_status_encoded'] = df['system_status'].map({'OK': 1, 'FAIL': 0})
```

**Gemini**

I have a telemetry dataset from satellites with the following columns: - temperature\_reading - fuel\_level - system\_status (OK/FAIL) - signal\_strength The dataset contains missing values, noise, and unrealistic readings. I want a Python script to preprocess it for predictive maintenance modeling. Requirements: 1. Fill missing numerical values appropriately. 2. Remove unrealistic values (e.g., negative fuel\_level). 3. Encode system\_status as numeric. 4. Scale all numerical features. 5. Split the dataset into training and testing sets (80/20 split). Generate a clean, ready-to-train dataset with Python code using pandas and sklearn. Include comments explaining each step.

✦ Certainly! I'll generate the Python code using pandas and scikit-learn to preprocess your telemetry dataset according to your requirements. I'll include comments for each step. For predictive maintenance, I'll assume system\_status is your target variable (y) and other columns are features (X).

Scale numerical features in the dataset  
Split the cleaned dataset into training and testing sets  
Generate Python code to train a predictive model

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes, so double-check it and use code with caution. [Learn more](#)

# 3. Encode system\_status as numeric

# Map 'OK' to 1 and 'FAIL' to 0

if 'system\_status' in df.columns:

df['system\_status\_encoded'] = df['system\_status'].map({'OK': 1, 'FAIL': 0})

# Drop the original 'system\_status' column if it's no longer needed as a feature

# We are assuming system\_status\_encoded will be our target variable 'y'

df = df.drop('system\_status', axis=1)

print("\n'system\_status' encoded to 'system\_status\_encoded' (1 for OK, 0 for FAIL).')

```
print("First 5 rows with encoded status:")
```

```
display(df.head())
```

else:

```
print("'system_status' column not found, skipping encoding.")
```

The screenshot shows a Jupyter Notebook environment with a code cell and its output. The code cell contains a conditional statement that checks if 'system\_status' is in the DataFrame columns. If it is, it encodes the values (OK to 1, FAIL to 0) and drops the original column. If not, it skips encoding. The output shows the first 5 rows of the DataFrame with columns: temperature\_reading, fuel\_level, signal\_strength, and system\_status\_encoded. To the right, the Gemini chat interface is visible, showing a user prompt about a telemetry dataset and a response offering to generate Python code for preprocessing.

```
# 3. Encode system_status as numeric
# Map 'OK' to 1 and 'FAIL' to 0
if 'system_status' in df.columns:
    df['system_status_encoded'] = df['system_status'].map({'OK': 1, 'FAIL': 0})
    # Drop the original 'system_status' column if it's no longer needed as a feature
    # We are assuming system_status_encoded will be our target variable 'y'
    df = df.drop('system_status', axis=1)
    print("\n'system_status' encoded to 'system_status_encoded' (1 for OK, 0 for FAIL).")
    print("First 5 rows with encoded status:")
    display(df.head())
else:
    print("'system_status' column not found, skipping encoding.")
```

|   | temperature_reading | fuel_level | signal_strength | system_status_encoded |
|---|---------------------|------------|-----------------|-----------------------|
| 0 | 30.306200           | 39.217205  | 16.048860       | 1                     |
| 1 | 52.677419           | 58.615157  | 32.910623       | 1                     |
| 2 | 34.920947           | 19.745123  | 38.201924       | 1                     |
| 3 | 28.688227           | 35.172150  | 40.975263       | 1                     |
| 4 | 56.071338           | 15.077102  | 17.994365       | 1                     |

```
# Define features (X) and target (y)
# Assuming 'system_status_encoded' is the target for predictive maintenance
if 'system_status_encoded' in df.columns:
    y = df['system_status_encoded']
    X = df.drop('system_status_encoded', axis=1)
    print("Features (X) and Target (y) defined.")
else:
    print("Target 'system_status_encoded' not found. Defining all remaining columns as features (X).")
    X = df.copy()
    y = pd.Series([0]*len(X)) # Create a dummy target if not found
```

```
# Define features (X) and target (y)
```

```
# Assuming 'system_status_encoded' is the target for predictive maintenance
```

```
if 'system_status_encoded' in df.columns:
```

```
    y = df['system_status_encoded']
```

```
    X = df.drop('system_status_encoded', axis=1)
```

```
    print("Features (X) and Target (y) defined.")
```

else:

```
    print("Target 'system_status_encoded' not found. Defining all remaining columns as features (X).")
```

```
    X = df.copy()
```

```
    y = pd.Series([0]*len(X)) # Create a dummy target if not found
```

```
# Identify numerical features for scaling
```

```
numerical_features = ['temperature_reading', 'fuel_level', 'signal_strength']
```

```
# Filter to ensure only existing columns are processed
```

```
numerical_features = [col for col in numerical_features if col in X.columns]
```

```
# Create a preprocessing pipeline for scaling numerical features
```

```
# We use ColumnTransformer to apply StandardScaler only to numerical features
```

```
preprocessor = ColumnTransformer(
```

```
    transformers=[
```

```
        ('num', StandardScaler(), numerical_features)
```

```
    ],
```

```
    remainder='passthrough' # Keep other columns (if any) as they are
```

```
)
```



```

# 4. Scale all numerical features
X_scaled = preprocessor.fit_transform(X)

# Convert the scaled array back to a DataFrame for easier handling
# Get feature names after preprocessing
scaled_feature_names =
preprocessor.named_transformers_['num'].get_feature_names_out(numerical_features)

# If there are other columns that were passed through, we need to get their names
# The remainder will be at the end of the X_scaled array
remainder_features = [col for col in X.columns if col not in numerical_features]

all_feature_names = list(scaled_feature_names) + remainder_features

X_processed = pd.DataFrame(X_scaled, columns=all_feature_names, index=X.index)

print("\nNumerical features scaled. First 5 rows of processed features (X_processed):")
display(X_processed.head())
print(f'Shape of processed features: {X_processed.shape}')

```

The screenshot shows a Jupyter Notebook environment. The left pane contains Python code for scaling numerical features. The right pane shows a Gemini AI chat interface with a user query and a response.

**Python Code in Jupyter Notebook:**

```

), remainder='passthrough' # Keep other columns (if any) as they are
)

# 4. Scale all numerical features
X_scaled = preprocessor.fit_transform(X)

# Convert the scaled array back to a DataFrame for easier handling
# Get feature names after preprocessing
scaled_feature_names = preprocessor.named_transformers_['num'].get_feature_names_out(numerical_features)

# If there are other columns that were passed through, we need to get their names
# The remainder will be at the end of the X_scaled array
remainder_features = [col for col in X.columns if col not in numerical_features]

all_feature_names = list(scaled_feature_names) + remainder_features

X_processed = pd.DataFrame(X_scaled, columns=all_feature_names, index=X.index)

print("\nNumerical features scaled. First 5 rows of processed features (X_processed):")
display(X_processed.head())
print(f'Shape of processed features: {X_processed.shape}')

```

**Gemini Chat Interface:**

**User Query:** I have a telemetry dataset from satellites with the following columns: - temperature\_reading - fuel\_level - system\_status (OK/FAIL) - signal\_strength. The dataset contains missing values, noise, and unrealistic readings. I want a Python script to preprocess it for predictive maintenance modeling. Requirements: 1. Fill missing numerical values appropriately. 2. Remove unrealistic values (e.g., negative fuel\_level). 3. Encode system\_status as numeric. 4. Scale all numerical features. 5. Split the dataset into training and testing sets (80/20 split). Generate a clean, ready-to-train dataset with Python code using pandas and sklearn. Include comments explaining each step.

**Gemini Response:** Certainly! I'll generate the Python code using pandas and scikit-learn to preprocess your telemetry dataset according to your requirements. I'll include comments for each step. For predictive maintenance, I'll assume system\_status is your target variable (y) and other columns are features (X).

**Buttons:** Scale numerical features in the dataset, Split the cleaned dataset into training and testing sets, Generate Python code to train a predictive model.

**Input Field:** What can I help you build?

**Model:** Gemini 2.5 Flash

```

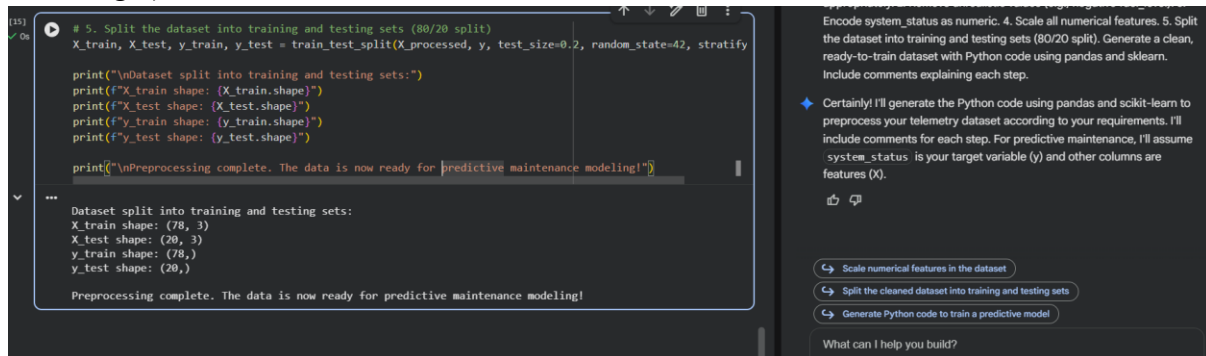
# 5. Split the dataset into training and testing sets (80/20 split)
X_train, X_test, y_train, y_test = train_test_split(X_processed, y, test_size=0.2,
random_state=42, stratify=y)

print("\nDataset split into training and testing sets:")
print(f'X_train shape: {X_train.shape}')
print(f'X_test shape: {X_test.shape}')

```

```
print(f'y_train shape: {y_train.shape}')
print(f'y_test shape: {y_test.shape}')
```

```
print("\nPreprocessing complete. The data is now ready for predictive maintenance modeling!")
```



The screenshot shows a Jupyter Notebook interface. The left pane contains a code cell with the following Python code:

```
# 5. Split the dataset into training and testing sets (80/20 split)
X_train, X_test, y_train, y_test = train_test_split(X_processed, y, test_size=0.2, random_state=42, stratify=y)

print("\nDataset split into training and testing sets:")
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")

print("\nPreprocessing complete. The data is now ready for predictive maintenance modeling!")
```

The right pane shows the output of the code cell, which is a text area containing the following text:

```
Dataset split into training and testing sets:
X_train shape: (78, 3)
X_test shape: (20, 3)
y_train shape: (78,)
y_test shape: (20,)

Preprocessing complete. The data is now ready for predictive maintenance modeling!
```

### Explanation:

- Missing values filled with median.
- Negative fuel levels removed.
- Status encoded as 0/1 for ML models.
- StandardScaler applied for predictive modeling.
- Dataset split into training and testing sets.

## **Task 4 – Online Gaming Performance Analytics**

### Scenario

An online gaming company collects player performance data:

- player\_score
- match\_duration
- region
- rank
- win\_status (Yes/No)

Data contains inconsistencies and duplicates.

### Task

Use AI to clean and preprocess gaming data.

### Instructions

- Remove duplicate player records
- Fill missing match\_duration values
- Encode categorical columns (region, win\_status)
- Standardize rank values (e.g., bronze, Bronze → BRONZE)
- Create a new feature: score\_per\_minute

### Expected Output

- Cleaned gaming dataset
- Standardized rank column
- New derived feature created
- Dataset ready for ranking analysis

**Prompt:**

I have an online gaming dataset with the following columns:

- player\_score
- match\_duration
- region
- rank (e.g., bronze, Bronze)
- win\_status (Yes/No)

The data has missing values, duplicates, and inconsistent ranks.

I want a Python script to clean and preprocess it for performance analysis.

Requirements:

1. Remove duplicate player records.
2. Fill missing match\_duration values.
3. Standardize rank values to uppercase.
4. Encode categorical columns (region, win\_status) numerically.
5. Create a new feature:  $\text{score\_per\_minute} = \text{player\_score} / \text{match\_duration}$ .

Provide Python code with comments explaining each step.

**Prompt:**

I have a climate dataset with the following columns:

- CO2\_emissions
- avg\_temperature
- country
- year
- extreme\_event (Yes/No)

The dataset has missing values and inconsistent country names.

I want a Python script to preprocess it for climate trend analysis.

Requirements:

1. Standardize country names (uppercase and remove spaces).
2. Handle missing CO2\_emissions and avg\_temperature values.
3. Encode extreme\_event as numeric.
4. Normalize numerical columns.
5. Generate a brief summary report showing:
  - Missing values before and after cleaning
  - Unique country count before and after standardization

Provide Python code with comments explaining each step.

**AI Generated Code:**

```
import pandas as pd
import numpy as np
```

```
from sklearn.preprocessing import OneHotEncoder
```

```
# Create a dummy DataFrame for demonstration purposes
```

```
data = {
    'player_id': range(100),
    'player_score': np.random.randint(100, 1000, 100),
    'match_duration': np.random.uniform(5, 60, 100),
    'region': np.random.choice(['NA', 'EU', 'AS', 'SA'], 100, p=[0.4, 0.3, 0.2, 0.1]),
    'rank': np.random.choice(['bronze', 'silver', 'Gold', 'PLATINUM', 'Bronze'], 100, p=[0.2,
0.25, 0.3, 0.15, 0.1]),
    'win_status': np.random.choice(['Yes', 'No', 'yes'], 100, p=[0.4, 0.5, 0.1])
}
df = pd.DataFrame(data)
```

```
# Introduce some missing values and duplicates for demonstration
```

```
df.loc[[5, 15, 25], 'match_duration'] = np.nan
```

```
df.loc[[10, 20], 'player_score'] = np.nan # Introduce missing player scores
```

```
# Add some duplicate player records
```

```
duplicate_rows = df.sample(5, random_state=42).copy()
```

```
duplicate_rows['player_id'] = [100, 101, 102, 103, 104] # Assign new IDs to avoid conflicts
during drop_duplicates
```

```
df = pd.concat([df, duplicate_rows], ignore_index=True)
```

```
print("Original DataFrame head with dummy data:")
```

```
display(df.head())
```

```
print("\nDataFrame Info:")
```

```
df.info()
```

The screenshot shows a Jupyter Notebook interface with a code cell and its output. The code cell contains the following Python code:

```
df.loc[[10, 20], 'player_score'] = np.nan # Introduce missing player scores

# Add some duplicate player records
duplicate_rows = df.sample(5, random_state=42).copy()
duplicate_rows['player_id'] = [100, 101, 102, 103, 104] # Assign new IDs to avoid conflicts during drop_duplicates
df = pd.concat([df, duplicate_rows], ignore_index=True)

print("Original DataFrame head with dummy data:")
display(df.head())
print("\nDataFrame Info:")
df.info()
```

The output of the code cell shows the first five rows of the DataFrame and its information:

|   | player_id | player_score | match_duration | region | rank     | win_status |
|---|-----------|--------------|----------------|--------|----------|------------|
| 0 | 0         | 307.0        | 45.045070      | EU     | silver   | yes        |
| 1 | 1         | 822.0        | 37.612142      | NA     | PLATINUM | No         |
| 2 | 2         | 646.0        | 20.959261      | NA     | PLATINUM | yes        |
| 3 | 3         | 184.0        | 54.228661      | AS     | bronze   | Yes        |
| 4 | 4         | 655.0        | 16.127461      | NA     | silver   | Yes        |

The DataFrame Info output is as follows:

```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 105 entries, 0 to 104
Data columns (total 6 columns):
 #   Column        Non-Null Count  Dtype  
---  --
 0   player_id     105 non-null    int64  
 1   player_score  103 non-null    float64
 2   match_duration 102 non-null    float64
 3   region        105 non-null    object  
 4   rank          105 non-null    object  
 5   win_status    105 non-null    object  
dtypes: float64(2), int64(1), object(3)
memory usage: 5.1+ KB
```

On the right side of the screenshot, there is a Gemini chat window. The user's message is:

I have an online gaming dataset with the following columns: - player\_score - match\_duration - region - rank (e.g., bronze, Bronze) - win\_status (Yes/No) The data has missing values, duplicates, and inconsistent ranks. I want a Python script to clean and preprocess it for performance analysis. Requirements: 1. Remove duplicate player records. 2. Fill missing match\_duration values. 3. Standardize rank values to uppercase. 4. Encode categorical columns (region, win\_status) numerically. 5. Create a new feature: score\_per\_minute = player\_score / match\_duration. Provide Python code with comments explaining each step.

The Gemini response is:

Certainly! I will provide a Python script to clean and preprocess your online gaming dataset according to your specifications. Since you haven't provided the actual dataset, I'll start by creating a dummy DataFrame that mimics your described columns, then proceed with the cleaning steps. The data cleaning and preprocessing for your online gaming dataset have been successfully completed using the dummy data. Here's a summary of the steps performed:

1. **Dummy Data Creation:** A DataFrame mimicking your online gaming data, including player scores, match durations, regions,

Below the response, there are three suggested actions:

- Summarize the processed DataFrame 'df'
- Analyze the new 'score\_per\_minute' feature
- Visualize the relationship between rank and score\_per\_minute

At the bottom of the chat window, there is a text input field with the placeholder "What can I help you build?" and a "Gemini 2.5 Flash" button.

```
# 1. Remove duplicate player records
# Assuming 'player_id' uniquely identifies a player, and we want to remove duplicate entries
# for the same player.
# If multiple rows can exist for a player (e.g., multiple matches), 'player_id' alone isn't
# enough to define a 'player record'.
# For this task, let's assume we want to remove rows that are exact duplicates across all
# columns.
initial_rows = len(df)
df = df.drop_duplicates()
print(f'Removed {initial_rows - len(df)} exact duplicate rows.')

# If 'player_id' should be unique and you want to keep only the first record for each
# player_id:
# df = df.drop_duplicates(subset=['player_id'], keep='first')
# print(f'Removed {initial_rows - len(df)} duplicate player_id records (keeping first).')

print("\nDataFrame head after removing duplicates:")
display(df.head())
```

**Data Cleaning and Preprocessing**

```
(17) ✓ Os
# 1. Remove duplicate player records
# Assuming 'player_id' uniquely identifies a player, and we want to remove duplicate entries for the same pl
# If multiple rows can exist for a player (e.g., multiple matches), 'player_id' alone isn't enough to define
# For this task, let's assume we want to remove rows that are exact duplicates across all columns.
initial_rows = len(df)
df = df.drop_duplicates()
print(f'Removed {initial_rows - len(df)} exact duplicate rows.')

# If 'player_id' should be unique and you want to keep only the first record for each player_id:
# df = df.drop_duplicates(subset=['player_id'], keep='first')
# print(f'Removed {initial_rows - len(df)} duplicate player_id records (keeping first).')

print("\nDataFrame head after removing duplicates:")
display(df.head())

... Removed 0 exact duplicate rows.

DataFrame head after removing duplicates:
  player_id  player_score  match_duration  region  rank  win_status
0         0           0.0         307.0    EU  silver        yes
1         1           1.0         822.0  NA PLATINUM        No
2         2           2.0         646.0  NA PLATINUM        yes
3         3           3.0         184.0    AS  bronze        Yes
4         4           4.0         655.0  NA  silver        Yes

(18) ✓ Os
# 2. Fill missing match_duration values
# Using the median for imputation as it's robust to outliers
median_duration = df['match_duration'].median()
df['match_duration'] = df['match_duration'].fillna(median_duration)
```

**Gemini**

I have an online gaming dataset with the following columns: - player\_score - match\_duration - region - rank (e.g., bronze, Bronze) - win\_status (Yes/No) The data has missing values, duplicates, and inconsistent ranks. I want a Python script to clean and preprocess it for performance analysis. Requirements: 1. Remove duplicate player records. 2. Fill missing match\_duration values. 3. Standardize rank values to uppercase. 4. Encode categorical columns (region, win\_status) numerically. 5. Create a new feature: score\_per\_minute = player\_score / match\_duration. Provide Python code with comments explaining each step.

Certainly! I will provide a Python script to clean and preprocess your online gaming dataset according to your specifications. Since you haven't provided the actual dataset, I'll start by creating a dummy DataFrame that mimics your described columns, then proceed with the cleaning steps. The data cleaning and preprocessing for your online gaming dataset have been successfully completed using the dummy data. Here's a summary of the steps performed:

- Dummy Data Creation:** A DataFrame mimicking your online gaming data, including player scores, match durations, regions,

Summarize the processed DataFrame 'df'  
Analyze the new 'score\_per\_minute' feature  
Visualize the relationship between rank and score\_per\_minute

What can I help you build?

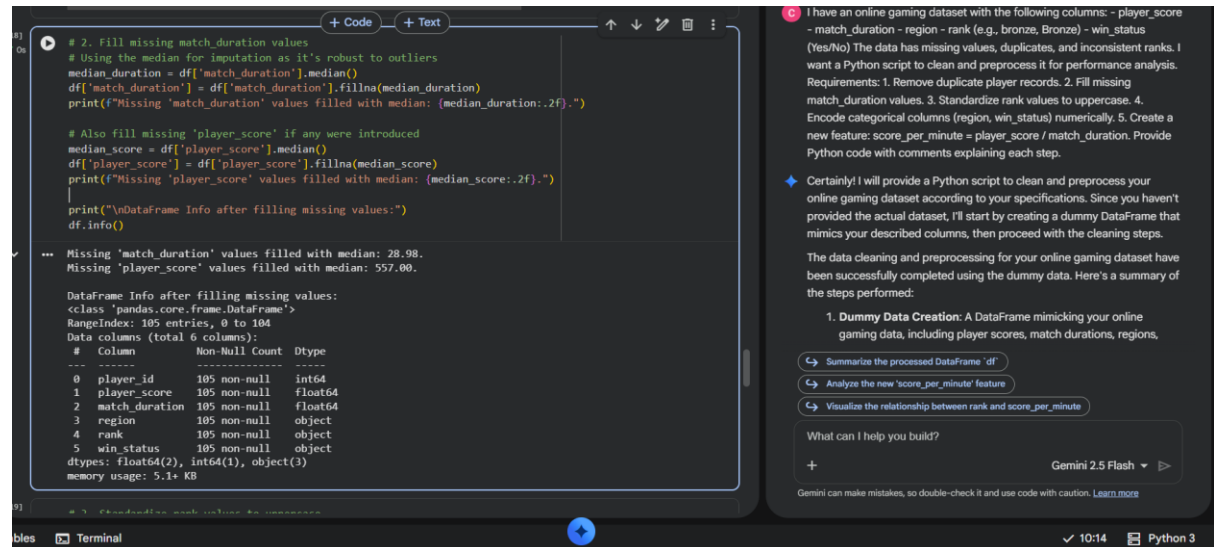
Gemini 2.5 Flash

Gemini can make mistakes, so double-check it and use code with caution. [Learn more](#)

```
# 2. Fill missing match_duration values
# Using the median for imputation as it's robust to outliers
median_duration = df['match_duration'].median()
df['match_duration'] = df['match_duration'].fillna(median_duration)
print(f'Missing 'match_duration' values filled with median: {median_duration:.2f}.')

# Also fill missing 'player_score' if any were introduced
median_score = df['player_score'].median()
df['player_score'] = df['player_score'].fillna(median_score)
print(f'Missing 'player_score' values filled with median: {median_score:.2f}.')
```

```
print("\nDataFrame Info after filling missing values:")
df.info()
```



```
# 2. Fill missing match_duration values
# Using the median for imputation as it's robust to outliers
median_duration = df['match_duration'].median()
df['match_duration'] = df['match_duration'].fillna(median_duration)
print(f"Missing 'match_duration' values filled with median: {median_duration:.2f}.")

# Also fill missing 'player_score' if any were introduced
median_score = df['player_score'].median()
df['player_score'] = df['player_score'].fillna(median_score)
print(f"Missing 'player_score' values filled with median: {median_score:.2f}.")
print("\nDataFrame Info after filling missing values:")
df.info()
```

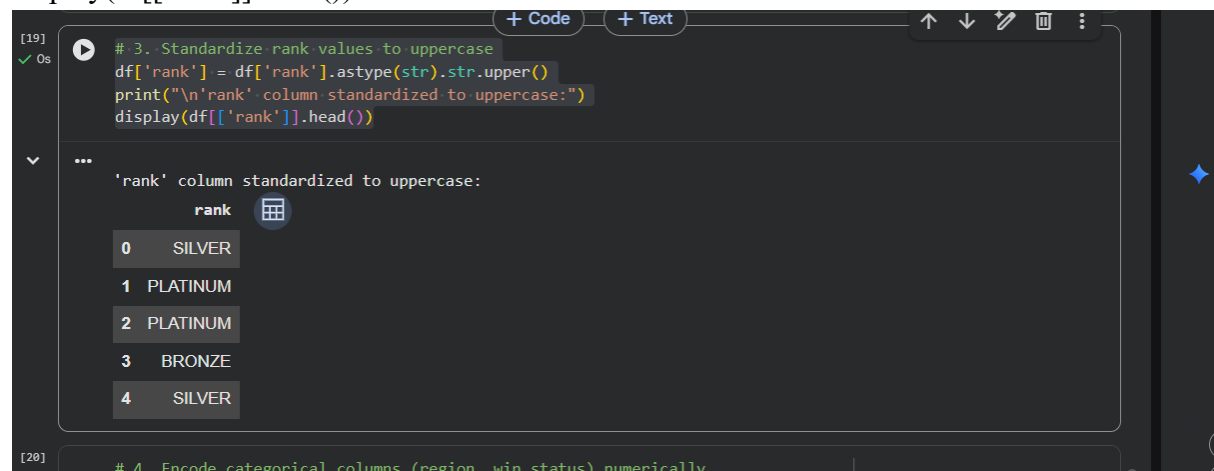
Missing 'match\_duration' values filled with median: 28.98.  
Missing 'player\_score' values filled with median: 557.00.

DataFrame Info after filling missing values:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 105 entries, 0 to 104
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   player_id   105 non-null    int64
 1   player_score 105 non-null    float64
 2   match_duration 105 non-null    float64
 3   region      105 non-null    object
 4   rank        105 non-null    object
 5   win_status  105 non-null    object
dtypes: float64(2), int64(1), object(3)
memory usage: 5.1+ KB
```

# 3. Standardize rank values to uppercase

```
df['rank'] = df['rank'].astype(str).str.upper()
print("\n'rank' column standardized to uppercase:")
display(df[['rank']].head())
```



```
# 3. Standardize rank values to uppercase
df['rank'] = df['rank'].astype(str).str.upper()
print("\n'rank' column standardized to uppercase:")
display(df[['rank']].head())
```

'rank' column standardized to uppercase:

|   | rank     |
|---|----------|
| 0 | SILVER   |
| 1 | PLATINUM |
| 2 | PLATINUM |
| 3 | BRONZE   |
| 4 | SILVER   |

# 4. Encode categorical columns (region, win\_status) numerically

# One-Hot Encoding for 'region' as there's no inherent order

```
encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
region_encoded = encoder.fit_transform(df[['region']])
region_df = pd.DataFrame(region_encoded,
columns=encoder.get_feature_names_out(['region']), index=df.index)
df = pd.concat([df, region_df], axis=1)
```

# Label Encoding for 'win\_status' (Yes/No) - mapping to 1/0

# Standardize to a consistent casing first

```
df['win_status'] = df['win_status'].str.upper()
df['win_status_encoded'] = df['win_status'].map({'YES': 1, 'NO': 0})
```

```
# Drop original categorical columns if not needed anymore
df = df.drop(columns=['region', 'win_status'])
```

```
print("\nDataFrame head after encoding categorical columns:")
display(df.head())
print("\nDataFrame Info after encoding:")
df.info()
```

The screenshot shows a Jupyter Notebook environment with the following code and output:

```
# Drop original categorical columns if not needed anymore
df = df.drop(columns=['region', 'win_status'])

print("\nDataFrame head after encoding categorical columns:")
display(df.head())
print("\nDataFrame Info after encoding:")
df.info()
```

**DataFrame head after encoding categorical columns:**

|   | player_id | player_score | match_duration | rank     | region_AS | region_EU | region_NA | region_SA | win_status |
|---|-----------|--------------|----------------|----------|-----------|-----------|-----------|-----------|------------|
| 0 | 0         | 307.0        | 45.045070      | SILVER   | 0.0       | 1.0       | 0.0       | 0.0       |            |
| 1 | 1         | 822.0        | 37.612142      | PLATINUM | 0.0       | 0.0       | 1.0       | 0.0       |            |
| 2 | 2         | 646.0        | 20.959261      | PLATINUM | 0.0       | 0.0       | 1.0       | 0.0       |            |
| 3 | 3         | 184.0        | 54.228661      | BRONZE   | 1.0       | 0.0       | 0.0       | 0.0       |            |
| 4 | 4         | 655.0        | 16.127461      | SILVER   | 0.0       | 0.0       | 1.0       | 0.0       |            |

**DataFrame Info after encoding:**

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 105 entries, 0 to 104
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   player_id              105 non-null    int64
1   player_score           105 non-null    float64
2   match_duration         105 non-null    float64
3   rank                   105 non-null    object
4   region_AS              105 non-null    float64
5   region_EU              105 non-null    float64
6   region_NA              105 non-null    float64
7   region_SA              105 non-null    float64
8   win_status_encoded     105 non-null    int64
dtypes: float64(6), int64(2), object(1)
memory usage: 7.5+ KB
```

The Gemini chat interface on the right provides the following context and suggestions:

I have an online gaming dataset with the following columns: - player\_score - match\_duration - region - rank (e.g., bronze, Bronze) - win\_status (Yes/No) The data has missing values, duplicates, and inconsistent ranks. I want a Python script to clean and preprocess it for performance analysis. Requirements: 1. Remove duplicate player records. 2. Fill missing match\_duration values. 3. Standardize rank values to uppercase. 4. Encode categorical columns (region, win\_status) numerically. 5. Create a new feature: score\_per\_minute = player\_score / match\_duration. Provide Python code with comments explaining each step.

Certainly! I will provide a Python script to clean and preprocess your online gaming dataset according to your specifications. Since you haven't provided the actual dataset, I'll start by creating a dummy DataFrame that mimics your described columns, then proceed with the cleaning steps.

The data cleaning and preprocessing for your online gaming dataset have been successfully completed using the dummy data. Here's a summary of the steps performed:

- Dummy Data Creation:** A DataFrame mimicking your online gaming data, including player scores, match durations, regions, and ranks.

Summary of the processed DataFrame 'df':

- Analyze the new 'score\_per\_minute' feature
- Visualize the relationship between rank and score\_per\_minute

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes, so double-check it and use code with caution. [Learn more](#)

```
# 5. Create a new feature: score_per_minute = player_score / match_duration
# Handle potential division by zero if 'match_duration' could be 0 (though our dummy data
won't have it)
df['score_per_minute'] = df['player_score'] / df['match_duration']
```

```
print("\nDataFrame head with new 'score_per_minute' feature:")
display(df.head())
```

```
print("\nPreprocessing complete. The DataFrame is now cleaned and prepared for
performance analysis!")
```



```
[21] 0s # 5. Create a new feature: score_per_minute = player_score / match_duration
# Handle potential division by zero if 'match_duration' could be 0 (though our dummy data won't have it)
df['score_per_minute'] = df['player_score'] / df['match_duration']

print("\nDataFrame head with new 'score_per_minute' feature:")
display(df.head())

print("\nPreprocessing complete. The DataFrame is now cleaned and prepared for performance analysis!")
```

... DataFrame head with new 'score\_per\_minute' feature:

|   | player_id | player_score | match_duration | rank     | region_AS | region_EU | region_NA | region_SA | win_status |
|---|-----------|--------------|----------------|----------|-----------|-----------|-----------|-----------|------------|
| 0 | 0         | 307.0        | 45.045070      | SILVER   | 0.0       | 1.0       | 0.0       | 0.0       |            |
| 1 | 1         | 822.0        | 37.612142      | PLATINUM | 0.0       | 0.0       | 1.0       | 0.0       |            |
| 2 | 2         | 646.0        | 20.959261      | PLATINUM | 0.0       | 0.0       | 1.0       | 0.0       |            |
| 3 | 3         | 184.0        | 54.228661      | BRONZE   | 1.0       | 0.0       | 0.0       | 0.0       |            |
| 4 | 4         | 655.0        | 16.127461      | SILVER   | 0.0       | 0.0       | 1.0       | 0.0       |            |

Preprocessing complete. The DataFrame is now cleaned and prepared for performance analysis!

### **Explanation:**

- Duplicate rows removed.
- Rank standardized and categorical columns encoded.
- Derived metric score\_per\_minute helps evaluate player performance efficiently.

## **Task 5 – Climate Change Research Dataset Preparation**

### **Scenario**

Researchers are analyzing global climate data containing:

- CO2\_emissions
- avg\_temperature
- country
- year
- extreme\_event (Yes/No)

The dataset contains inconsistent country names and missing values.

### **Task**

Use AI to preprocess the dataset for climate trend modeling.

### **Instructions**

- Standardize country names
- Handle missing CO2\_emissions and avg\_temperature
- Encode extreme\_event
- Normalize numerical columns
- Generate a brief summary report showing:
  - o Missing values before vs after cleaning
  - o Unique country count before vs after standardization

### **Expected Output**

- Clean and standardized climate dataset
- Encoded categorical variables
- Normalized numerical features



- Data quality summary report

### **AI Generated Code:**

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

# Create a dummy DataFrame for demonstration purposes
data = {
    'CO2_emissions': np.random.rand(100) * 1000 + 500, # 500-1500
    'avg_temperature': np.random.rand(100) * 15 + 10, # 10-25
    'country': np.random.choice(['USA', 'usA ', 'Canada', 'france ', 'Germany', 'France'], 100,
p=[0.25, 0.15, 0.2, 0.1, 0.15, 0.15]),
    'year': np.random.randint(1990, 2023, 100),
    'extreme_event': np.random.choice(['Yes', 'No'], 100, p=[0.3, 0.7])
}
df_climate = pd.DataFrame(data)

# Introduce some missing values for demonstration
df_climate.loc[[5, 15, 25], 'CO2_emissions'] = np.nan
df_climate.loc[[10, 20, 30], 'avg_temperature'] = np.nan

print("Original DataFrame head with dummy data:")
display(df_climate.head())
print("\nDataFrame Info:")
df_climate.info()

# Record initial state for summary report
initial_missing_values = df_climate.isnull().sum()
initial_unique_countries = df_climate['country'].nunique()
```

```

# Introduce some missing values for demonstration
df_climate.loc[[5, 15, 25], 'CO2_emissions'] = np.nan
df_climate.loc[[10, 20, 30], 'avg_temperature'] = np.nan

print("Original DataFrame head with dummy data:")
display(df_climate.head())
print("\nDataFrame Info:")
df_climate.info()

# Record initial state for summary report
initial_missing_values = df_climate.isnull().sum()
initial_unique_countries = df_climate['country'].nunique()

Original DataFrame head with dummy data:
  CO2_emissions  avg_temperature  country  year  extreme_event
0      693.939150      19.481227      USA    1999             No
1     1191.258718      19.975101  Germany    2003             No
2      623.125407      14.412592   Canada    2012             Yes
3      877.634617      18.448436      USA    2011             Yes
4      814.692138      24.327858   France    2021             No

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 5 columns):
 #   Column                Non-Null Count  Dtype  
---  --
 0   CO2_emissions         97 non-null    float64
 1   avg_temperature       97 non-null    float64
 2   country               100 non-null   object  
 3   year                 100 non-null   int64   
 4   extreme_event         100 non-null   object  
dtypes: float64(2), int64(1), object(2)
memory usage: 4.0+ KB

```

# 1. Standardize country names (uppercase and remove spaces)

```

df_climate['country'] = df_climate['country'].astype(str).str.upper().str.strip()
print("\n'country' column standardized:")
display(df_climate[['country']].head())
print(f"Unique countries after standardization: {df_climate['country'].nunique()}")

```

```

# 1. Standardize country names (uppercase and remove spaces)
df_climate['country'] = df_climate['country'].astype(str).str.upper().str.strip()
print("\n'country' column standardized:")
display(df_climate[['country']].head())
print(f"Unique countries after standardization: {df_climate['country'].nunique()}")

'country' column standardized:
  country
0      USA
1  GERMANY
2  CANADA
3      USA
4  FRANCE

Unique countries after standardization: 4

```

# 2. Handle missing CO2\_emissions and avg\_temperature values

# Using median for imputation as it's robust to outliers

```

median_co2 = df_climate['CO2_emissions'].median()
df_climate['CO2_emissions'] = df_climate['CO2_emissions'].fillna(median_co2)
print(f"Missing 'CO2_emissions' values filled with median: {median_co2:.2f}.")

```

```

median_temp = df_climate['avg_temperature'].median()
df_climate['avg_temperature'] = df_climate['avg_temperature'].fillna(median_temp)
print(f"Missing 'avg_temperature' values filled with median: {median_temp:.2f}.")

```

```

print("\nDataFrame Info after filling missing values:")

```

```
df_climate.info()
```

```
[24] # 2. Handle missing CO2 emissions and avg temperature values
# Using median for imputation as it's robust to outliers
median_co2 = df_climate['CO2_emissions'].median()
df_climate['CO2_emissions'] = df_climate['CO2_emissions'].fillna(median_co2)
print(f"Missing 'CO2_emissions' values filled with median: {median_co2:.2f}.")

median_temp = df_climate['avg_temperature'].median()
df_climate['avg_temperature'] = df_climate['avg_temperature'].fillna(median_temp)
print(f"Missing 'avg_temperature' values filled with median: {median_temp:.2f}.")

print("\nDataFrame Info after filling missing values:")
df_climate.info()
```

... Missing 'CO2\_emissions' values filled with median: 958.01.  
Missing 'avg\_temperature' values filled with median: 17.19.

DataFrame Info after filling missing values:  
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 100 entries, 0 to 99  
Data columns (total 5 columns):  
# Column Non-Null Count Dtype  
--- --  
0 CO2\_emissions 100 non-null float64  
1 avg\_temperature 100 non-null float64  
2 country 100 non-null object  
3 year 100 non-null int64  
4 extreme\_event 100 non-null object  
dtypes: float64(2), int64(1), object(2)  
memory usage: 4.0+ KB

```
# 3. Encode extreme_event as numeric (Yes=1, No=0)
```

```
df_climate['extreme_event_encoded'] = df_climate['extreme_event'].map({'Yes': 1, 'No': 0})
```

```
# Drop the original 'extreme_event' column if not needed
```

```
df_climate = df_climate.drop(columns=['extreme_event'])
```

```
print("\n'extreme_event' column encoded to 'extreme_event_encoded':")
```

```
display(df_climate.head())
```

```
[25] # 3. Encode extreme_event as numeric (Yes=1, No=0)
df_climate['extreme_event_encoded'] = df_climate['extreme_event'].map({'Yes': 1, 'No': 0})
# Drop the original 'extreme_event' column if not needed
df_climate = df_climate.drop(columns=['extreme_event'])
print("\n'extreme_event' column encoded to 'extreme_event_encoded':")
display(df_climate.head())
```

... 'extreme\_event' column encoded to 'extreme\_event\_encoded':

|   | CO2_emissions | avg_temperature | country | year | extreme_event_encoded |
|---|---------------|-----------------|---------|------|-----------------------|
| 0 | 693.939150    | 19.481227       | USA     | 1999 | 0                     |
| 1 | 1191.258718   | 19.975101       | GERMANY | 2003 | 0                     |
| 2 | 623.125497    | 14.412592       | CANADA  | 2012 | 1                     |
| 3 | 877.634617    | 18.448436       | USA     | 2011 | 1                     |
| 4 | 814.692138    | 24.327858       | FRANCE  | 2021 | 0                     |

```
# 4. Normalize numerical columns (CO2_emissions, avg_temperature)
```

```
scaler = StandardScaler()
```

```
numerical_cols = ['CO2_emissions', 'avg_temperature']
```

```
df_climate[numerical_cols] = scaler.fit_transform(df_climate[numerical_cols])
```

```
print("\nNumerical columns normalized:")
```

```
display(df_climate.head())
```

```
[26] # 4. Normalize numerical columns (CO2_emissions, avg_temperature)
scaler = StandardScaler()
numerical_cols = ['CO2_emissions', 'avg_temperature']
df_climate[numerical_cols] = scaler.fit_transform(df_climate[numerical_cols])

print("\nNumerical columns normalized:")
display(df_climate.head())

...
Numerical columns normalized:
   CO2_emissions  avg_temperature  country  year  extreme_event_encoded
0      -0.943849      0.555558     USA    1999                        0
1       0.781665      0.682045  GERMANY    2003                        0
2      -1.189547     -0.742576   CANADA    2012                        1
3      -0.306494      0.291049     USA    2011                        1
4      -0.524881      1.796834   FRANCE    2021                        0

[27] # 5. Generate a brief summary report
```

# 5. Generate a brief summary report

```
print("\n--- Summary Report ---")
```

```
print("\nMissing values before cleaning:")
print(initial_missing_values)
```

```
print("\nMissing values after cleaning:")
print(df_climate.isnull().sum())
```

```
print(f"\nUnique country count before standardization: {initial_unique_countries}")
print(f"Unique country count after standardization: {df_climate['country'].nunique()}")
```

```
print("\nPreprocessing complete. The DataFrame is now cleaned and prepared for climate trend analysis!")
```

```
[27] # 5. Generate a brief summary report

print("\n--- Summary Report ---")

print("\nMissing values before cleaning:")
print(initial_missing_values)

print("\nMissing values after cleaning:")
print(df_climate.isnull().sum())

print(f"\nUnique country count before standardization: {initial_unique_countries}")
print(f"Unique country count after standardization: {df_climate['country'].nunique()}")

print("\nPreprocessing complete. The DataFrame is now cleaned and prepared for climate trend analysis!")

...
--- Summary Report ---

Missing values before cleaning:
CO2_emissions    3
avg_temperature    3
country           0
year              0
extreme_event     0
dtype: int64

Missing values after cleaning:
CO2_emissions    0
avg_temperature    0
country           0
year              0
extreme_event     0
dtype: int64

Unique country count before standardization: 6
Unique country count after standardization: 4

Preprocessing complete. The DataFrame is now cleaned and prepared for climate trend analysis!
```

## Explanation:

- Country names standardized to uppercase and stripped of extra spaces.

- Missing values handled with median for robustness.
- Categorical encoding for extreme events.
- Numerical features normalized.
- Data quality summary report included for Word submission.